



**GURU GOBIND SINGH FOUNDATION'S
GURU GOBIND SINGH COLLEGE OF ENGINEERING AND RESEARCH CENTRE, NASHIK**

Certificate

This is to certify that Mr./Miss _____ of Second Year Artificial Intelligence and Data Science, Roll No.:____ University Examination, Seat Number_____has completed all the practical work/Term work in Subject_____satisfactorily in ***Department of Artificial Intelligence and Data Science*** as prescribed by the ***Savitribai Phule Pune University*** in the academic year 20__- 20__.

Practical In-charge

Head of the Department

Principal

Index

Practical No.	Name of Experiment	Date of Performance	Date of Completion	Grade/Marks	Sign
1	Real estate agents want help to predict the house price for regions in the USA. He gave you the dataset to work on and you decided to use the Linear Regression Model. Create a model that will help him to estimate what the house would sell for. URL for a dataset: https://github.com/huzaifsayed/Linear-Regression-Model-for-House-PricePrediction/blob/master/USA_Housing.csv				
2	Build a Multiclass classifier using the CNN model. Use MNIST or any other suitable dataset. a. Perform Data Pre-processing b. Define Model and perform training c. Evaluate Results using confusion matrix				
3	Design RNN or its variant including LSTM or GRU a) Select a suitable time series dataset. Example – predict sentiments based on product reviews b) Apply for prediction				
4	Design and implement a CNN for Image Classification a) Select a suitable image classification dataset (medical imaging, agricultural, etc.). b) Optimized with different hyper-parameters including learning rate, filter size, no. of layers, optimizers, dropouts, etc.				
5	Design and implement Deep Convolutional GAN to generate images of faces/digits from a set of given images				

Date of Submission:

Practical In-charge

ASSIGNMENT: 1

TITLE

House Price Prediction using Linear Regression (USA Housing Dataset)

OBJECTIVES

- To understand the concept of Linear Regression.
- To build a predictive model for house price estimation.
- To analyze the relationship between housing features and price.
- To implement Linear Regression using Python and Scikit-learn.
- To evaluate model performance using regression metrics.

PROBLEM STATEMENT

Real estate agents require an efficient system to predict house prices based on various features such as income, number of rooms, and population.

In this experiment, a **Linear Regression model** is developed using the USA Housing dataset to estimate house prices.

Dataset Link:

https://github.com/huzaihsayed/Linear-Regression-Model-for-House-PricePrediction/blob/master/USA_Housing.csv

OUTCOMES

After completing this experiment, students will be able to:

- Understand regression-based predictive modeling.
- Preprocess real-world datasets.
- Train and test a Linear Regression model.
- Evaluate performance using metrics like MSE and R^2 .
- Interpret relationships between features and target variable.

SOFTWARE AND HARDWARE REQUIREMENTS

Software:

- Python 3.x
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Jupyter Notebook / VS Code

Hardware:

- Minimum 4GB RAM
- Intel i3 or higher processor

DATASET DESCRIPTION

Dataset Name: USA Housing Dataset

Source: GitHub

Dataset Link:

https://github.com/huzaihsayed/Linear-Regression-Model-for-House-PricePrediction/blob/master/USA_Housing.csv

Total Records: ~5000

Important Features:

- Avg. Area Income
- Avg. Area House Age
- Avg. Area Number of Rooms
- Avg. Area Number of Bedrooms
- Area Population

Target Variable:

- Price (House price)

THEORY

Linear Regression

Linear Regression is a supervised learning algorithm used for predicting continuous values.

It models the relationship between independent variables (features) and dependent variable (target).

Mathematics

Multiple Linear Regression:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Where:

- y = Predicted house price
- x_1, x_2 = Features
- β_0 = Intercept
- β_1, β_2 = Coefficients
- ε = Error

Assumptions of Linear Regression

- Linearity
- Independence
- Homoscedasticity
- Normal distribution of errors

Cost Function

Mean Squared Error (MSE):

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$$

Evaluation Metrics

1. Mean Squared Error (MSE)
2. Root Mean Squared Error (RMSE)
3. R^2 Score

Applications

- Real estate price prediction
- Sales forecasting
- Economic modeling

ALGORITHM

1. Start
2. Import required libraries
3. Load dataset
4. Perform data preprocessing
5. Select features and target variable
6. Split dataset into training and testing sets
7. Train Linear Regression model
8. Predict house prices
9. Evaluate model performance
10. Stop

FLOWCHART

Draw your flowchart

CODING CONCEPTS USED

1. Data Loading
`pd.read_csv()`
2. Data Visualization
`seaborn.pairplot()`
`matplotlib.pyplot.scatter()`
3. Data Splitting
`train_test_split()`
4. Model Training
`LinearRegression()`
`fit()`
5. Prediction
`predict()`
6. Evaluation
`mean_squared_error()`
`r2_score()`

TEST CASES

Test Case 1

Input: Dataset loaded successfully

Expected Output: ~5000 records

Test Case 2

Train-Test split

Expected Output: 80% training, 20% testing

Test Case 3

Model prediction

Expected Output: Predicted house prices

TEST DATA SET

Dataset Used: USA Housing Dataset

Records: ~5000

Features: 5

Target Variable: Price

MATHEMATICS

Regression Equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

MSE:

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$$

RMSE:

$$RMSE = \sqrt{MSE}$$

ANALYSIS

- Higher income areas generally have higher house prices.
- Number of rooms significantly impacts house price.
- Population density also influences price.
- The model shows good predictive performance with proper feature selection.

CONCLUSION

In this experiment, a Linear Regression model was implemented to predict house prices using the USA Housing dataset. The model successfully captured relationships between housing features and prices. Evaluation metrics confirmed that the model provides reliable predictions. This experiment demonstrates the practical application of regression models in real estate and business analytics.

MCQs

1. Linear Regression is used for:
 - A) Classification
 - B) Regression
 - C) Clustering
 - D) SortingAnswer:

2. The target variable in this dataset is:

- A) Income
- B) Rooms
- C) Price
- D) Population

Answer:

3. MSE measures:

- A) Accuracy
- B) Error
- C) Probability
- D) Classification

Answer:

4. R^2 score indicates:

- A) Error
- B) Model fit
- C) Data size
- D) Sorting

Answer:

5. Linear Regression predicts:

- A) Categories
- B) Continuous values
- C) Images
- D) Text

Answer:

ASSIGNMENT: 2

TITLE

Multiclass Image Classification using Convolutional Neural Network (CNN) with MNIST Dataset

OBJECTIVES

- To understand the concept of Convolutional Neural Networks (CNN).
- To build a multiclass classifier for handwritten digit recognition.
- To perform data preprocessing on image datasets.
- To train a CNN model and evaluate its performance.
- To analyze results using a confusion matrix.

PROBLEM STATEMENT

Image classification is a key application of Deep Learning. In this experiment, build a **multiclass classifier** using a **Convolutional Neural Network (CNN)** to classify handwritten digits (0–9) from the MNIST dataset.

Tasks to be performed:

- a) Perform Data Preprocessing
- b) Define CNN Model and perform training
- c) Evaluate results using confusion matrix

Dataset: MNIST Dataset (available in TensorFlow/Keras)

OUTCOMES

After completing this experiment, students will be able to:

- Understand CNN architecture and its components.
- Perform preprocessing on image datasets.
- Train deep learning models for multiclass classification.
- Evaluate model performance using confusion matrix.
- Apply CNNs in computer vision tasks.

SOFTWARE AND HARDWARE REQUIREMENTS

Software:

- Python 3.x
- NumPy
- TensorFlow / Keras
- Matplotlib
- Seaborn
- Scikit-learn
- Jupyter Notebook / VS Code

Hardware:

- Minimum 4GB RAM
- Intel i3 or higher processor

DATASET DESCRIPTION

Dataset Name: MNIST Dataset

Source: TensorFlow/Keras

Total Images: 70,000

- Training: 60,000 images
- Testing: 10,000 images

Image Size: 28×28 pixels

Classes: 10 (Digits 0 to 9)

Each image represents a handwritten digit.

THEORY

Convolutional Neural Network (CNN)

CNN is a deep learning model used for image processing and classification.

Main components:

- Convolution Layer
- Pooling Layer
- Fully Connected Layer

Convolution Operation

Convolution extracts features from images using filters.

Feature map = Input Image * Kernel

Pooling Layer

Pooling reduces dimensionality.

Types:

- Max Pooling
- Average Pooling

Activation Functions

ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

Softmax (for output layer):

$$\text{Softmax}(x_i) = e^{x_i} / \sum e^{x_j}$$

Loss Function

Categorical Cross Entropy:

$$\text{Loss} = - \sum y_i \log(p_i)$$

Confusion Matrix

A confusion matrix evaluates classification performance.

It shows:

- True Positives
- False Positives
- True Negatives
- False Negatives

ALGORITHM

1. Start
2. Import required libraries
3. Load MNIST dataset
4. Normalize image data
5. Reshape data for CNN input
6. Define CNN model
7. Compile model
8. Train model
9. Predict test data
10. Generate confusion matrix
11. Stop

FLOWCHART

Draw your flowchart

CODING CONCEPTS USED

1. Dataset Loading
`mnist.load_data()`
2. Data Preprocessing
Normalization
Reshaping
3. CNN Layers
`Conv2D()`
`MaxPooling2D()`
`Flatten()`
`Dense()`
4. Model Training
`fit()`
5. Evaluation
`confusion_matrix()`
`classification_report()`

TEST CASES

Test Case 1

Input: Dataset loaded

Expected Output: 70,000 images

Test Case 2

Model training

Expected Output: Increasing accuracy

Test Case 3

Prediction

Expected Output: Correct digit classification

TEST DATA SET

Dataset Used: MNIST Dataset

Training Images: 60,000

Testing Images: 10,000

Classes: 10

MATHEMATICS

Convolution:

$$\text{Feature Map} = \text{Input} \otimes \text{Kernel}$$

Softmax:

$$\text{Softmax}(x_i) = e^{x_i} / \sum e^{x_j}$$

Loss:

Categorical Cross Entropy

Accuracy:

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

ANALYSIS

- CNN effectively extracts spatial features from images.
- Pooling reduces computation complexity.
- High accuracy achieved for digit classification.
- Confusion matrix helps identify misclassified digits.

CONCLUSION

In this experiment, a Convolutional Neural Network was implemented to classify handwritten digits using the MNIST dataset. The model achieved high accuracy and demonstrated the effectiveness of CNNs in image classification tasks. The confusion matrix provided insights into model performance and misclassifications.

MCQs

1. CNN is mainly used for:

- A) Text processing
- B) Image processing
- C) Networking
- D) Encryption

Answer:

2. The MNIST dataset contains:

- A) Text data
- B) Audio data
- C) Handwritten digits
- D) Videos

Answer:

3. Softmax function is used for:

- A) Regression
- B) Binary classification
- C) Multiclass classification
- D) Clustering

Answer:

4. Pooling layer is used to:

- A) Increase size
- B) Reduce dimensions
- C) Delete data
- D) Encrypt data

Answer:

5. Confusion matrix is used to:

- A) Train model
- B) Evaluate classification
- C) Load data
- D) Normalize data

Answer:

ASSIGNMENT: 3

TITLE

Sequential Data Prediction using Recurrent Neural Networks (RNN/LSTM/GRU) – Sentiment Analysis on Product Reviews

OBJECTIVES

- To understand Recurrent Neural Networks (RNN) and their variants (LSTM, GRU).
- To apply deep learning models on sequential/text data.
- To perform sentiment analysis on product reviews.
- To preprocess textual data for neural networks.
- To train and evaluate an LSTM/GRU model for prediction.

PROBLEM STATEMENT

Sequential data such as text requires models that can capture dependencies between words. In this experiment, design and implement a **Recurrent Neural Network (RNN)** or its variants (**LSTM/GRU**) to perform **sentiment analysis on product reviews**.

Tasks:

- a) Select a suitable dataset (e.g., product reviews dataset)
 - b) Preprocess the text data
 - c) Build and train an RNN/LSTM/GRU model
 - d) Predict sentiment (Positive/Negative)
- Example Dataset: Amazon Product Reviews / IMDB Reviews Dataset

OUTCOMES

After completing this experiment, students will be able to:

- Understand sequential data processing using RNNs.
- Implement LSTM/GRU models for NLP tasks.
- Preprocess and tokenize text data.
- Train deep learning models for sentiment classification.
- Evaluate prediction performance.

SOFTWARE AND HARDWARE REQUIREMENTS

Software:

- Python 3.x
- NumPy
- Pandas
- TensorFlow / Keras
- Matplotlib
- Scikit-learn
- Jupyter Notebook / VS Code

Hardware:

- Minimum 4GB RAM
- Intel i3 or higher processor

DATASET DESCRIPTION

Dataset Name: Product Reviews Dataset / IMDB Dataset

Typical Features:

- Review Text
- Sentiment Label (Positive / Negative)

Classes:

- 0 → Negative
- 1 → Positive

Purpose:

Used for training models to classify sentiment based on textual data.

THEORY

Sequential Data

Sequential data is data where order matters.

Examples:

- Text
- Time series
- Speech signals

Recurrent Neural Network (RNN)

RNN is designed to process sequences by maintaining a memory of previous inputs.

Equation:

$$h_t = f(Wx_t + Uh_{t-1} + b)$$

Limitations of RNN

- Vanishing gradient problem
- Difficulty in learning long-term dependencies

Long Short-Term Memory (LSTM)

LSTM solves RNN limitations using memory cells.

Components:

- Forget Gate
- Input Gate
- Output Gate

Gated Recurrent Unit (GRU)

GRU is a simplified version of LSTM with fewer gates:

- Update Gate
- Reset Gate

Text Preprocessing

Steps:

- Tokenization
- Vocabulary creation
- Sequence padding
- Encoding words into numbers

Loss Function

Binary Cross Entropy:

$$\text{Loss} = - [y \log(p) + (1 - y) \log(1 - p)]$$

ALGORITHM

1. Start
2. Import required libraries
3. Load dataset
4. Preprocess text (tokenization, padding)
5. Split dataset into training and testing sets
6. Build LSTM/GRU model
7. Compile model
8. Train model
9. Predict sentiment
10. Evaluate performance
11. Stop

FLOWCHART

Draw your flowchart

CODING CONCEPTS USED

1. Dataset Loading
`pd.read_csv()`
2. Text Processing
`Tokenizer()`
`pad_sequences()`
3. Model Building
`Embedding()`
`LSTM() / GRU()`
`Dense()`

4. Model Training

`fit()`

5. Evaluation

`evaluate()`

TEST CASES

Test Case 1

Input: Sample text data

Expected Output: Tokenized sequences

Test Case 2

Model training

Expected Output: Increasing accuracy

Test Case 3

Prediction

Expected Output: Correct sentiment classification

TEST DATA SET

Dataset Used: Product Reviews Dataset

Classes:

- Positive
- Negative

MATHEMATICS

RNN Equation:

$$h_t = f(Wx_t + Uh_{t-1} + b)$$

Sigmoid Function:

$$P(y=1) = 1 / (1 + e^{-x})$$

Loss Function:

Binary Cross Entropy

ANALYSIS

- LSTM captures long-term dependencies in text.
- Tokenization converts text into numerical form.
- Padding ensures equal sequence length.
- GRU provides faster training compared to LSTM.

CONCLUSION

In this experiment, an RNN-based model using LSTM was implemented for sentiment analysis on product reviews. The model successfully classified reviews into positive and negative categories. LSTM and GRU models are powerful tools for handling sequential data and are widely used in Natural Language Processing tasks.

MCQs

1. RNN is used for:

- A) Image data
- B) Sequential data
- C) Sorting
- D) Encryption

Answer:

2. LSTM solves:

- A) Overfitting
- B) Vanishing gradient problem
- C) Data loss
- D) Noise

Answer:

3. GRU is:

- A) More complex than LSTM
- B) Simpler than LSTM
- C) Not used
- D) A CNN

Answer:

4. Tokenization is used for:

- A) Image processing
- B) Text processing
- C) Audio processing
- D) Sorting

Answer:

5. Binary cross entropy is used for:

- A) Regression
- B) Classification
- C) Clustering
- D) Sorting

Answer:

ASSIGNMENT: 4

TITLE

Design and Implementation of CNN for Image Classification with Hyperparameter Optimization

OBJECTIVES

- To understand the working of Convolutional Neural Networks (CNN).
- To perform image classification using deep learning.
- To select and work with a real-world image dataset (medical/agricultural).
- To optimize CNN performance using different hyperparameters.
- To evaluate model performance and improve accuracy.

PROBLEM STATEMENT

Image classification is a core task in computer vision with applications in healthcare, agriculture, and industry. In this experiment, design and implement a **CNN model** to classify images from a suitable dataset (e.g., plant disease detection or medical imaging dataset).

Tasks:

- a) Select a suitable dataset
- b) Perform preprocessing
- c) Build CNN model
- d) Optimize model using hyperparameters such as:
 - Learning rate
 - Number of layers
 - Filter size
 - Optimizer
 - Dropout rate

- e) Evaluate model performance

Example Dataset Options:

- Plant Disease Dataset
- Chest X-ray Dataset
- CIFAR-10 Dataset

OUTCOMES

After completing this experiment, students will be able to:

- Understand CNN architecture and components.
- Implement image classification models.
- Perform hyperparameter tuning.
- Improve model performance using optimization techniques.
- Evaluate deep learning models effectively.

SOFTWARE AND HARDWARE REQUIREMENTS

Software:

- Python 3.x
- TensorFlow / Keras
- NumPy
- Matplotlib
- Scikit-learn
- Jupyter Notebook / VS Code

Hardware:

- Minimum 4GB RAM
- GPU (optional but recommended)
- Intel i3 or higher processor

DATASET DESCRIPTION

Example Dataset: CIFAR-10 Dataset

Total Images: 60,000

Training Images: 50,000

Testing Images: 10,000

Image Size: 32×32 pixels

Classes (10):

- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

Dataset is used for multiclass image classification tasks.

THEORY

Convolutional Neural Network (CNN)

CNN is a deep learning model used for processing image data.

Components:

- Convolution Layer
- Pooling Layer
- Fully Connected Layer

Convolution Operation

Extracts features from images using filters (kernels).

Feature Map = Input $\otimes$ Kernel

Pooling Layer

Reduces spatial dimensions and computation.

Types:

- Max Pooling
- Average Pooling

Dropout

Dropout is used to reduce overfitting by randomly disabling neurons.

Hyperparameter Optimization

Hyperparameters affect model performance.

Examples:

- Learning Rate
- Number of Filters
- Kernel Size
- Number of Layers
- Batch Size
- Optimizer (Adam, SGD)
- Dropout Rate

Loss Function

Categorical Cross Entropy:

Loss = $-\sum y_i \log(p_i)$

ALGORITHM

1. Start
2. Import required libraries
3. Load dataset
4. Normalize image data
5. Build CNN model
6. Apply hyperparameter tuning
7. Compile model
8. Train model
9. Evaluate performance
10. Compare results
11. Stop

FLOWCHART

Draw your flowchart

CODING CONCEPTS USED

1. Dataset Loading
`cifar10.load_data()`
2. Data Preprocessing
Normalization
3. CNN Layers
`Conv2D()`
`MaxPooling2D()`
`Dropout()`
`Flatten()`
`Dense()`
4. Model Training
`fit()`
5. Hyperparameter Tuning
Changing layers, filters, optimizer

TEST CASES

Test Case 1

Input: Dataset loaded successfully

Expected Output: 60,000 images

Test Case 2

Normalization

Expected Output: Pixel values between 0 and 1

Test Case 3

Hyperparameter tuning

Expected Output: Improved accuracy

TEST DATA SET

Dataset Used: CIFAR-10 Dataset

Training Samples: 50,000

Testing Samples: 10,000

Classes: 10

MATHEMATICS

Convolution:

$$\text{Feature Map} = \text{Input} \otimes \text{Kernel}$$

Softmax:

$$P(y=i) = e^{x_i} / \sum e^{x_j}$$

Loss:

Categorical Cross Entropy

ANALYSIS

- Increasing layers improves feature extraction.
- Dropout reduces overfitting.
- Learning rate affects convergence speed.
- Adam optimizer generally performs better than SGD.
- Hyperparameter tuning improves model accuracy.

CONCLUSION

In this experiment, a CNN model was designed and implemented for image classification. Various hyperparameters such as learning rate, number of layers, and dropout were tuned to improve model performance. The optimized model achieved better accuracy, demonstrating the importance of hyperparameter tuning in deep learning applications.

MCQs

1. CNN is mainly used for:
A) Text processing
B) Image processing
C) Sorting
D) Encryption
Answer:
2. Dropout is used to:
A) Increase data
B) Reduce overfitting
C) Sort data
D) Normalize data
Answer:
3. CIFAR-10 dataset contains:
A) Text
B) Images
C) Audio
D) Video
Answer:
4. Adam optimizer is used for:
A) Data cleaning
B) Model training
C) Sorting
D) Encryption
Answer:
5. Learning rate controls:
A) Data size
B) Speed of training
C) Number of classes
D) Image size
Answer:

ASSIGNMENT: 5

TITLE

Design and Implementation of Deep Convolutional Generative Adversarial Network (DCGAN) for Image Generation

OBJECTIVES

- To understand the concept of Generative Adversarial Networks (GANs).
- To design and implement a Deep Convolutional GAN (DCGAN).
- To generate realistic images of digits or faces from random noise.
- To understand adversarial training between generator and discriminator.
- To evaluate the quality of generated images.

PROBLEM STATEMENT

Image generation is an important task in deep learning where models learn to create new data similar to the training dataset.

In this experiment, implement a **Deep Convolutional GAN (DCGAN)** to generate images such as:

- Handwritten digits (MNIST dataset)
- Human faces (CelebA dataset)

The model consists of:

- Generator → Generates fake images
- Discriminator → Distinguishes real vs fake images

The goal is to train the model so that generated images become realistic.

OUTCOMES

After completing this experiment, students will be able to:

- Understand GAN architecture and working principles.
- Implement generator and discriminator networks.
- Train adversarial models effectively.
- Generate synthetic images from random noise.
- Evaluate generative models qualitatively.

SOFTWARE AND HARDWARE REQUIREMENTS

Software:

- Python 3.x
- TensorFlow / Keras
- NumPy
- Matplotlib
- Jupyter Notebook / VS Code

Hardware:

- Minimum 4GB RAM
- GPU recommended for faster training
- Intel i3 or higher processor

DATASET DESCRIPTION

Option 1: MNIST Dataset

- 70,000 grayscale images
- Image size: 28×28
- Classes: Digits (0–9)

Option 2: CelebA Dataset

- Large dataset of celebrity faces
- RGB images
- Used for face generation

THEORY

Generative Adversarial Networks (GAN)

GAN consists of two neural networks:

1. Generator (G)
2. Discriminator (D)

They compete with each other:

- Generator tries to fool discriminator
- Discriminator tries to detect fake images

Objective Function

Minimax function:

$$\min_G \max_D V(D, G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

Generator Network

- Takes random noise as input
- Produces synthetic images

Discriminator Network

- Classifies images as real or fake
- Outputs probability (0 or 1)

Deep Convolutional GAN (DCGAN)

DCGAN uses convolutional layers for better image generation.

Key features:

- Conv2DTranspose (Generator)
- Conv2D (Discriminator)
- Batch Normalization
- LeakyReLU activation

Activation Functions

- ReLU (Generator)
- LeakyReLU (Discriminator)
- Sigmoid (Output)

ALGORITHM

1. Start
2. Import required libraries
3. Load dataset (MNIST or CelebA)
4. Normalize images
5. Build generator model
6. Build discriminator model
7. Combine models into GAN
8. Train discriminator on real and fake images
9. Train generator to fool discriminator
10. Repeat for multiple epochs
11. Generate new images
12. Stop

FLOWCHART

Draw your flowchart

CODING CONCEPTS USED

1. Neural Network Layers
 - `Conv2D()`
 - `Conv2DTranspose()`
 - `Dense()`
2. Activation Functions
 - `ReLU()`
 - `LeakyReLU()`
 - `Sigmoid()`
3. Model Training
 - `train_on_batch()`
4. Random Noise Generation
 - `np.random.normal()`

TEST CASES

Test Case 1

Input: Dataset loaded

Expected Output: Images normalized

Test Case 2

Generator output

Expected Output: Random noise converted into image

Test Case 3

Training

Expected Output: Improved image quality over epochs

TEST DATA SET

Dataset Used: MNIST Dataset

Images: 70,000

Image Size: 28×28

MATHEMATICS

GAN Objective:

$\min_G \max_D V(D, G)$

Loss Function:

Binary Cross Entropy

Generator tries to minimize $\log(1 - D(G(z)))$

Discriminator tries to maximize $\log D(x)$

ANALYSIS

- GAN learns to generate realistic images over time.
- Generator improves as discriminator becomes stronger.
- Training is unstable and requires tuning.
- DCGAN produces better image quality than basic GAN.

CONCLUSION

In this experiment, a Deep Convolutional GAN was implemented to generate images from random noise. The generator successfully learned to produce realistic images of digits by competing against the discriminator. DCGAN demonstrates the power of deep learning in generative modeling and has applications in image synthesis, data augmentation, and creative AI.

MCQs

1. GAN consists of:
 - A) One network
 - B) Two networks
 - C) Three networks
 - D) No networks

Answer:

2. Generator is used to:
- A) Classify data
 - B) Generate fake data
 - C) Sort data
 - D) Store data

Answer:

3. Discriminator is used to:
- A) Generate images
 - B) Detect fake images
 - C) Train data
 - D) Normalize data

Answer:

4. DCGAN uses:
- A) Linear layers
 - B) Convolution layers
 - C) Decision trees
 - D) SVM

Answer:

5. GAN training is:
- A) Simple
 - B) Stable
 - C) Adversarial
 - D) Linear

Answer: